

به نام خدا

دراین کد سه روش مختلف برای حل سودوکو استفاده شده:

1. **CSP + Arc Consistency (AC-3)**
2. **Backtracking + Forward Checking + MRV**
3. **Local Search : Min-Conflicts**

این برنامه ابتدا جدول را با سه روش از کاربر دریافت می کند: 1. چند سودوکو از قبل مشخص شده 2. ورود دستی با کپیبرد 3. خواندن از فایل ، سپس یکی از سه الگوریتم را برای حل انتخاب می کند. در نهایت در جواب جدول حل شده، زمان اجرا و میزان حافظه مصرف شده نمایش داده می شود. همچنین اگر کاربر بخواهد میتواند مراحل کار را هم مرحله به مرحله نمایش دهد. همچنین این کد دارای بررسی صحت ورودی، جلوگیری از خطاهای فایل، و اندازه گیری زمان و حافظه مصرفی است.

کاربر میتواند هر زمان که بخواهد از برنامه خارج شود یا به منوی قبلی برگردد. کاربر میتواند انتخاب کند که راه حل برای اون به نمایش در آید یا فقط جواب آخر باشد.

در این کد، در سودوکو هر خانه جدول یک متغیر، دامنه آن اعداد 1 تا 9، و قیود شامل عدم تکرار عدد در سطر، ستون و مربع 3×3 هستند.

الگوریتم های جست و جو:

الگوریتم اول ("AC-3" CSP + Arc Consistency) : ابتدا با انتشار قید دامنه ها را کوچک می کند و سپس در صورت نیاز از جستجوی بازگشتی با روش MRV استفاده می کند. این روش معمولاً سریع و دقیق است.

الگوریتم دوم (Backtracking + Forward Checking + MRV) : یک جستجوی بازگشتی است که پس از هر مقداری، دامنه همسایه ها را بررسی می کند تا از ایجاد تناقض جلوگیری شود.

الگوریتم سوم (Min-Conflicts) : یک روش جستجوی محلی است که با مقداری اولیه تصادفی شروع کرده و با کم کردن تعداد خطاها سعی در رسیدن به جواب دارد. این روش سریع است ولی تضمین حل ندارد.

توضیح خلاصه درمورد هر تابع :

cross

تولید ترکیب همه اعضای دو رشته برای ساخت نام خانه‌های جدول.

gridValues

تبدیل رشته ۸۱ کاراکتری ورودی به دیکشنری خانه‌ها و مقادیر آن‌ها.

display

چاپ جدول سودوکو به شکل مرتب و قابل خواندن در خروجی.

visualizeStep

نمایش مرحله‌ای جدول هنگام اجرای الگوریتم‌ها برای مشاهده روند حل.

solveCspAC3

حل سودوکو با روش CSP همراه با Arc Consistency و جستجوی بازگشتی.

assign

اختصاص یک مقدار به یک خانه و حذف سایر مقادیر از دامنه آن.

remoove

حذف یک مقدار از دامنه یک خانه و اعمال انتشارقید به همسایه‌ها.

searchCsp

جستجوی بازگشتی با استفاده از MRV برای تکمیل حل پس از AC3.

solveBacktrackingFC

حل سودوکو با Backtracking به همراه Forward Checking و MRV.

forwardCheck

حذف مقدار اختصاص داده شده از دامنه همسایه‌ها و بررسی تناقض.

backtrackFC

جستجوی بازگشتی با انتخاب خانه دارای کمترین دامنه و بررسی مرحله‌ای.

solve_min_conflicts

حل سودوکو با روش جستجوی محلی Min-Conflicts و کاهش تدریجی خطاها.

count_total_conflicts

محاسبه تعداد کل تضادها در ستون‌ها و مربع‌های 3×3 .

score_cell

محاسبه تعداد تضادهای یک خانه خاص در ستون و مربع مربوطه.

validate_input

بررسی معتبر بودن ورودی کاربر و امکان خروج از برنامه.

get_input_puzzle

دریافت جدول سودوکو از کاربر از طریق انتخاب آماده، ورود دستی یا فایل.

get_algorithm_choice

دریافت انتخاب الگوریتم از کاربر.

format_memory

تبدیل مقدار حافظه از بایت به واحد مناسب (KB) یا (MB).

main

تابع اصلی برنامه برای مدیریت ورودی، اجرای الگوریتم، نمایش نتیجه و تکرار برنامه.

تحلیل پیچیدگی همین کد:

تعداد متغیرها $\leq 9 \times 9$ ----- اندازه دامنه هر متغیر ≤ 9

1. CSP + AC3

این الگوریتم یک روش جستجوی هوشمند هست که قبل از حدس زدن، تا حد ممکن با منطق دامنه‌ها را کوچک می‌کند. اول با AC3 اعداد غیرممکن را از خانه‌ها حذف می‌کند (انتشار قید). بعد اگر جدول کامل حل نشد، با جستجوی بازگشتی و انتخاب خانه با کمترین گزینه (MRV) ادامه می‌دهد.

پیچیدگی:

در بدترین حالت نمایی است (مثل همه جستجوهای کامل)، ولی چون قبل از جستجو دامنه‌ها را کم می‌کند، معمولاً سریع‌تر از بقیه روش‌های کامل است. حافظه تقریباً به اندازه تعداد خانه‌ها است.

زمان: نمایی $O(n^{(n^2)})$ در بدترین حالت و حافظه $O(n^2)$

2. Backtracking + Forward Checking

این الگوریتم جستجوی بازگشتی ساده‌تر با یک بررسی سریع بعد از هر مقداردهی. یک خانه را انتخاب می‌کند، یک عدد امتحان می‌کند، بعد آن عدد را از همسایه‌ها حذف می‌کند. (Forward Checking) اگر تناقض ایجاد شود، برمی‌گردد عقب.

پیچیدگی:

در بدترین حالت نمایی است. معمولاً کمی کندتر از AC3 چون استنتاج ضعیف‌تری دارد. حافظه تقریباً به اندازه تعداد خانه‌ها است.

زمان: نمایی $O(n^{n^2})$ و حافظه: $O(n^2)$

3. Min-Conflicts

این الگوریتم یک روش جستجوی محلی (بدون بازگشت). اول یک جدول تقریباً کامل تصادفی می‌سازد. بعد با جابجا کردن اعداد سعی می‌کند تعداد خطاها را کم کند تا به صفر برسد.

پیچیدگی:

زمان تقریباً خطی نسبت به تعداد تکرارها. $(\max_steps)$ حافظه کم است. اما تضمین نمی‌کند همیشه جواب پیدا کند.

زمان $O(\max_steps \times n)$ و حافظه $O(n^2)$